

CSBB 254

Mini Project Report

PrintEase

Submitted By

Gufran Alam (241210042)

Harshit Jhamb (241210047)

Kavitharkika Abhiram (241210058)

Krish Karandikar (241210065)

Submitted to: Dr. Sahil



DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING
NATIONAL INSTITUTE OF TECHNOLOGY
DELHI

Table of Contents

1 Abstract	5
2 Introduction	5
2.1 Problem statement	5
2.2 Proposed solution	5
2.3 Scope	5
3 Software Requirements Specification	6
3.1. Introduction	6
3.1.1 Definitions, Acronyms, and Abbreviations	7
3.2. Overall Description	7
3.2.1 Product Perspective	8
3.2.2 Product Functions	8
3.2.3 User Classes and Characteristics	9
3.2.4 Operating Environment	9
3.2.5 Design and Implementation Constraints	10
3.2.6 Assumptions and Dependencies	10
3.3 Functional Requirements	11
3.3.1 File Upload and Management	11
3.3.2 General User Dashboard	11
3.3.3 Natural Language Instruction Processing	12
3.3.4 Payment Processing	13
3.3.5 Print Job Creation and Management	13
3.3.6 Queue Management System	14
3.3.7 Admin Dashboard	14
3.3.8 Back Up System Requirements (Future Scope)	15
3.4 External Interfaces Requirements	15
3.4.1 User Interface	15
3.4.2 Hardware Interface	16
3.4.3 Software Interface	16
3.4.4 Communication Interface	16
3.5 Non Functional Requirements	16
3.5.1 Performance	16
3.5.2 Reliability	17
3.5.3 Security	17
3.5.4 Usability	17
3.5.5 Scalability	18
4 Software Design Description	18

4.1 Introduction	18
4.1.1 Definitions, Acronyms, and Abbreviations	18
4.2 System Architecture	19
4.2.1 Decomposition Description	20
4.2.2 Design Rationale	24
4.3 Data Description	25
4.4 Component Interaction	26
4.5 User Interface	27
4.6 Project Schedule	28
5 Implementation	28
5.1 Tech stack	28
5.1.1 Frontend Technologies	29
5.1.2 Backend Technologies	29
5.1.3 Security and Authentication	29
5.1.4 Payment Integration	29
5.2 Key Algorithms/ Logic	30
5.2.1 Authentication Logic	30
5.2.2 File Upload Logic	30
5.2.3 Print Cost Calculation Logic	30
5.2.4 Payment Processing Logic	30
5.2.5 Request Management Logic	31
5.4 Product Screenshots	32
5.4.1 Login Page	32
5.4.2 Home Dashboard	33
5.4.3 PDF Upload Section	34
5.4.4 Print Details Page	34
5.4.5 My Orders Page	35
5.4.6 Admin Dashboard	35
5.4.7 Order Summary Screen for Admin	36
5.4.8 Payment Portal	37
6. Testing	38
6.1 Testing Strategies	38
6.2 Testing Cases	38
7. Conclusion And Future Scope	40
7.1 Conclusion	40
7.2 Future Enhancements	40
8. References and Bibliography	41

Table of Figures

Figure	Title	Page No.
1	Collaboration Diagram	19
2	Level 0 Data Flow Diagram	20
3	Level 1 Data Flow Diagram	21
4.1	Level 2 Data Flow Diagram-File Upload and Document Processing	22
4.2	Level 2 Data Flow Diagram-Cost Calculation Engine	22
4.3	Level 2 Data Flow Diagram-Print Job Generation	23
4.4	Level 2 Data Flow Diagram- Admin Dashboard and Queue Monitoring	24
5	Entity Relationship Diagram	25
6	Sequence diagram	26
7	Use case diagram	27
8	Gantt Chart	28
9	Tables in Database	31
10	Order Sample as received by the admin	31
11	Stored files in the DB	32
12	Users in the DB example	32
13	Login Page	33
14	User Dashboard	33
15	Upload Docs	34
16	Print details	34
17	My orders	35
18	Admin Dashboard	36
19	Order Summary	37
20	Payment Portal	37

1 Abstract

PrintEase is a digital ecosystem developed using the MERN stack to modernize and streamline the printing infrastructure at educational institutions. The system addresses current bottlenecks—such as lost files and long wait times—by providing a unified platform for remote document submission, automated cost calculation, and secure payment processing. Our key features include a Natural Language Processing (NLP) engine that interprets plain-text printing instructions (e.g., "print only the last page in color") and a First-In-First-Out (FIFO) cloud queue for organized job execution. By integrating external APIs like Razorpay for payments and Google Drive for cloud storage, PrintEase provides students with structured access to printing services while offering administrators a centralized dashboard for real-time queue management and financial tracking.

2 Introduction

2.1 Problem statement

The existing manual printing process in colleges is highly inefficient, relying on student emails and hand-written logs, which frequently leads to lost documents and a complete lack of transparency in the printing queue. These issues are heightened during peak academic periods, such as end-of-semester lab submissions, where the absence of a structured system results in bottlenecks and physical wait times extending into several hours. Furthermore, the manual handling of financial transactions and print configurations increases the risk of human error and administrative overhead for print room staff.

2.2 Proposed solution

The web-based printing system we propose replaces this process with a unified digital platform that allows users to upload documents, specify printing instructions using dropdown menus and natural language from customized printing, make payments, and queue print jobs for execution. This system will provide students with a structured and remote access to the printing system, which will drastically reduce physical wait times whilst offering the administrators a centralized dashboard to manage high-volume print queues and financial transactions, reducing the chances of system breakdown during rush hours.

2.3 Scope

The proposed printing system allows users to submit print jobs remotely through a web interface. Users can upload files directly from local storage or via cloud storage platforms (like Google

Drive). To simplify the user experience, the system utilizes drop down Menus for simple print jobs and provides a section that uses a Natural Language Processing Engine to interpret printing instructions for custom print jobs. For example, users can provide plain text commands such as “2nd and last page in color,” which the system parses into structured print instructions, while taking into account the instructions interpreted from the dropdown menus.

Once the document has been uploaded and instructions have been processed, the system triggers an automated cost calculation system based on a pre-defined rate for paper size, amount of paper used, and ink type (B/W or color). The system integrates Razorpay’s API connection to provide a secure Dynamic Payment Gateway, including methods such as United Payments Interface (UPI), Net Banking, Credit/Debit Cards, and mobile wallets.

After a successful transaction, the job enters a centralized database where information regarding the print job and the user is stored before the job moves on to the Cloud Queue. This queue follows a First-In-First-Out (FIFO) logic to ensure fairness. This Queue can be monitored by an Administrative Dashboard. This dashboard allows print room staff to view the real-time status of all paid jobs. After the print has taken place, the User can come and take their printouts based on the print ID produced by the System.

Thus, by automating the transition from payment to printing, the system eliminates manual logging and prevents print jobs from going missing due to being lost in the mail, whilst increasing printing efficiency.

3 Software Requirements Specification

3.1. Introduction

The purpose of this software requirement specification (SRS) document is to provide a comprehensive definition of the functional and non-functional requirements, system constraints, and acceptance criteria for PrintEase, a web-based automated printing system tailored for college environments such as ours. This document serves as a technical blueprint for developers, project evaluators, and stakeholders, ensuring a shared understanding of the system's core functionalities. This document will describe the operational functionalities of the printing system whilst providing a rationale for the choices made during the design and development process.

3.1.1 Definitions, Acronyms, and Abbreviations

Term	Description
SRS	Software Requirement Specification
NLP	Natural Language Processing
B/W	Black and White
MERN	MongoDB, Express.js, React, Node.js
API	Application Programming Interface
UPI	United Payments Interface
FIFO	First in First Out
UI	User Interface
LAN	Local Area Network
HTTPS	HyperText Transfer Protocol Secure
SSL	Secure Sockets Layer
PDF	Portable Document Format
DOCX	Open Office XML Document
ID	Identification Document
CSV	Comma-Separated Values
FR	Functional Requirements
NR	Non-Functional Requirements
IR	External-Interface Requirements

3.2. Overall Description

This section provides high-level context on how our proposed system will interact with real-life infrastructure, its users, and the technical boundaries it must operate in.

3.2.1 Product Perspective

PrintEase is a standalone digital ecosystem designed to replace the existing manual, email-based workflow of the printing system in our college. It acts as a middleman between the college's digital users and the physical printing hardware. The system will be built using the MERN (MongoDB, Express.js, React, Node.js) stack for frontend and backend services, ensuring high performance and scalability.

The system's relationship with external components and internal modules is defined by the following interactions or interfaces

- User Interface(UI): A web-based application that can be accessed through desktop and mobile devices, allowing for remote job submissions
- Software Interfaces: The backend interacts with dedicated Python-based NLP modules to interpret specialized print instructions, Cloud storage APIs such as Google Drive to enable file uploading, and User verification, RazorPay API for secure payment processing
- Hardware Interfaces: The system communicates with the physical printer, transmitting the document to be printed and instructions over a local area network (LAN)

3.2.2 Product Functions

The major functions of the system include:

- User Authentication: Provides secure login for students and staff, whilst giving a specialized dashboard for administrators
- Profile Management: Allows Users to manage document history, track current job status, and view past transactions
- Multi-Source Document Upload: Allows users to upload files from local storage or integrated cloud service APIs such as Google Drive
- Hybrid Print Instruction Configuration: print instructions are generated by interpreting standard dropdown menus for common print jobs and natural language instructions, which are interpreted by a Python-based NLP module
- Cost Estimation: Performs real-time cost calculations based on chosen fields
- Secure Payment Processing: Provides secure and multi-method payment options through RazorPay payment gateway (UPI, Net Banking, Credit/Debit Cards, and mobile wallets)

- Persistent Database Storage: utilizes a MongoDB backend to store comprehensive records for every print job
- Automated Job Queuing: Maintains a payment-verified, digital print queue(FIFO) to ensure organized job execution and prevent document loss
- Administrative monitoring: provides a dedicated dashboard for Admins to see the active queue, monitor printer hardware status, and manage the physical execution of jobs

3.2.3 User Classes and Characteristics

General Users:

This includes students and faculty who currently get printouts from the existing printing system in college. They are required to have basic technical knowledge of web browsers and mobile payment apps, and a basic understanding of common UI icons and styles. They are responsible for uploading documents, providing print instructions, and completing payments. They also utilize the application to track job status in real time.

Print Operators:

This includes college employees responsible for the day-to-day operations of the print room. They are required to have moderate technical knowledge regarding computer Systems and printer hardware maintenance. They utilize the admin-dedicated dashboard to manage print queues, resolve any hardware-related issues, and manually trigger print jobs if the automated system is not fully operational.

3.2.4 Operating Environment

- Web Browsers: PrintEase will be accessible via any modern standards-compliant web browser(Google Chrome, Microsoft Edge, Mozilla Firefox, etc.) on both desktop and mobile operating systems
- Backend Server: Will run on a Node.js runtime environment, facilitating the MERN stack's data handling and API routing, as well as integrated NLP modules
- Database: A Local MongoDB database which will act as a primary persistent storage for user data and transaction logs

- Network Connected Printers: The application as a whole requires a stable internet connection to enable communication between the web server, RazorPay payment gateway, and physical printing hardware

3.2.5 Design and Implementation Constraints

- Security Standards: The system must utilize HTTPS/SSL (HyperText Transfer Protocol Secure/Secure Sockets Layer) to ensure secure data transmission
- File format Limitations: The printer is restricted to only accept file formats such as PDF and DOCX
- Connectivity Dependencies: The system is entirely dependent on stable internet connectivity. Any disruption will result in the temporary suspension of job submissions or payment failures
- Payment Gateway Integration: the implementation must adhere to the specific API rate limits, authentication protocols, and webhook structures provided by Razorpay
- Printer hardware limitations: The software's output is ultimately constrained by the physical printer available in the college. If the printer is not internet-connected, it would require an additional local desktop application that automates the process between the job queue on the cloud and the actual printer.

3.2.6 Assumptions and Dependencies

Assumptions

- User Readiness: We first assume that all General Users and Print Operators have access to a web-enabled device. Furthermore, we assume that General Users possess valid payment methods described in section 3.2.2
- Hardware: We assume that the college printers are fully functional and operational, and connected to the LAN during operational hours
- Infrastructure Reliability: We assume that the college provides a stable power supply to the local database host and printer
- NLP Model: We assume that the used NLP module is pre-trained and optimized

Dependencies

- External Service Uptime: Payment Processing and cloud document retrieval is entirely dependent on the operational status of third-party APIs

3.3 Functional Requirements

This section provides a technical breakdown of exactly how the system will execute its primary functions

3.3.1 File Upload and Management

- File upload: The system shall provide a "Drag-and-Drop" or file-explorer interface to allow users to upload documents directly from their local hardware (Laptop or Mobile).
- Cloud Service Integration: The system shall interface with third-party APIs (specifically Google Drive) to allow users to browse and select documents for import without requiring a local download
- File Format Validation: The system shall strictly validate all uploads; it shall only accept PDF and DOCX file formats. If a user attempts to upload an unsupported format, the system shall return an error message and block the submission.
- File Size Limit: The system shall enforce a maximum file size limit (50mb) per upload to prevent server memory exhaustion and ensure manageable transmission speeds over the college network.
- Metadata Extraction: Upon successful upload, the system shall automatically extract metadata, including page count and document dimensions, to be used for cost estimation.
- Secure Temporary Storage: The system shall store uploaded files in a secure, temporary directory or cloud bucket. These files shall be encrypted at rest and held only until the print job is successfully executed or the session expires, after which they shall be purged to ensure data privacy.

3.3.2 General User Dashboard

- Real Time status tracking: The system shall allow users to track their job's progress through a personal dashboard. The status must reflect one of the following states
- Printer Availability: The system shall display the current printer status to inform users of potential delays before they submit a job.
- Personal Print Analytics: The dashboard shall provide a summary of the user's total interaction with the system including total orders placed, completed prints, pending prints

- Financial Overview: The system shall display the Total money spent by the user till date
- Transaction History: Users shall have access to a chronological log of all past transactions. Each entry must show the Job ID, date, amount paid, and provide a link to download the digital receipt
- Notification/Alert: when the job becomes complete status a, notification/alert will be sent on the dashboard to inform the User, Similar notification/alert will be sent when the handoff is completed

3.3.3 Natural Language Instruction Processing

- Input Capture: The system shall provide a dual-input interface consisting of standard UI controls (dropdowns/checkboxes) and a dedicated text field for natural language instructions.
- Intent Classification: The system shall utilize a Python-based NLP model to classify the user's intent, distinguishing between general print requests and specific "Intelligent Overrides" (e.g., identifying that "only page 5 in color" is a specialized instruction).
- Parameter Extraction: The engine shall extract key variables from the text input including
 - Quantity
 - Scope: specific page ranges(eg:"first 10 pages" or "last page only")
 - Ink used: specific pages or section requiring color or B/W print
 - Layout: orientation of print and if single sided or back-to-back print is required
- Logical Override Protocol: The system shall prioritize NLP-extracted parameters over conflicting dropdown selections. If a user selects "B/W" in a dropdown but writes "Print the cover in color," the system shall dynamically update the configuration to reflect the specialized text instruction.
- Instruction Mapping: The system shall convert the interpreted natural language instructions into a structured JSON object compatible with the backend print server and database schema.
- User Verification: Before proceeding to payment, the system shall display a "Final Configuration Summary" derived from the NLP processing, allowing the user to confirm the system correctly understood their text instructions.

3.3.4 Payment Processing

- **Dynamic Cost Calculation:** The system shall calculate the total job cost in real time by applying a pricing algorithm to the metadata extracted from the document as well as from the dropdown Menus/checkboxes and NLP module. The formula shall account for
 - No of pages in B/W print
 - No of pages in Color print

Selecting single sided would take the total amount of pages and then apply the formula whereas back to back would half the number of pages rounding upwards. The algorithm will account for any specialized instructions generated through the NLP

- **RazorPay gateway integration:** The system shall initiate a secure payment session via the Razorpays API, providing the user with a "Checkout" interface that supports UPI, Debit/Credit Cards, and Net Banking
- **Transaction ID generation:** For every transaction, the system shall generate a unique Transaction ID. This ID must be linked to the specific Print Job ID in the database to ensure accurate financial tracking.
- **Real-time Verification:** The system shall utilize asynchronous webhooks to listen for a "Payment Success" signal from the bank. The job shall remain in a "Pending Payment" state and hidden from the print queue until this confirmation is received
- **Transaction Status Feedback:** Upon payment completion, the system shall display an immediate success or failure notification to the user. In the event of a failed or cancelled transaction, the system shall allow the user to retry the payment without re-uploading the document.
- **Digital Receipts:** The system shall generate a digital receipt for every successful transaction, accessible to the user via their profile history for future reference

3.3.5 Print Job Creation and Management

- **Unique Job ID generation:** The system shall automatically generate a non-repeating, Unique Job ID for every new submission. This ID shall serve as the primary key across the database, payment gateway, and admin dashboard
- **Data Persistence:** The system shall store a complete "job" in the MongoDB database which must include User ID, Job ID, Total amount, Print status, Payment Status, Payment ID, and file metadata

- **User Verification:** The system shall link each Job ID to a specific user profile. This allows the Print Administrator to verify the student's identity via the dashboard during the physical handoff of the printed document.
- **Job status Tracking:** The system shall maintain a "Status" field for every job, which must update in real-time as it moves through the following stages: Paid/Queued, Processing, Completed, or Cancelled.
- **Document Retention Policy:** To aid in quick document fetching, the database shall retain job metadata (history of uploaded documents) for a short period of time (eg. 1 month), but the system shall automatically delete the associated source files from temporary storage once the status is marked as completed

3.3.6 Queue Management System

- **Automated Queue Entry:** The system shall automatically move a job into the active queue immediately upon receiving a "Success" webhook from the payment gateway
- **Scheduling Logic:** The system shall implement a FIFO scheduling algorithm by default, using the payment confirmation timestamp as the primary sorting key to ensure fairness
- **Administrator Overrides:** The system shall provide an interface for administrators to manually adjust queue priority (e.g., moving an urgent faculty document to the top of the list) when necessary

3.3.7 Admin Dashboard

- **Live Queue Visualization:** The dashboard shall display a real time list of all payment verified jobs, sorted by their current status
- **Execution Control:** prevent document mixing during rush hours, the system shall allow Admins to manually trigger the start of a print job. Admins shall also have the authority to pause or cancel specific jobs in the event of hardware malfunctions or user errors. The admins will also be able to change the status of the printer to offline/online during the transition between operational and non operational hours
- **Print Verification and Search:** The dashboard shall include a Search and Verification module. Admins shall be able to search by Job ID or Student Name to verify the owner of a document before the physical handoff, preventing unauthorized collections

- Transaction and Handoff Logging: The system shall record a timestamped entry whenever an admin marks a job as "Handed Over," providing a complete audit trail from initial upload to final collection.
- Dashboard Information and Search: The system shall provide an interface through which the admin will be able to see Total earnings of that day and have the ability to open a dropdown menu which will allow them to download a CSV file which contains transaction or print history. This process will essentially query information from the database based on selected fields

3.3.8 Back Up System Requirements (Future Scope)

In the event of failure of hardware or software integration with the entire proposed systems the following applications or processes could be adopted to develop and deploy PrintEase effectively but are not prioritized in the development cycle.

- Local Bridge Application: In the event that a printer cannot maintain a direct persistent connection to the web-based server or does not have internet connectivity hardware at all, the system can be supported by a Local Desktop Bridge Application. This application will act as a local relay, fetching queued jobs from the cloud database via a secure polling mechanism and transmitting them to the printer over the local area network (USB)
- Automated User Verification: In the event that physical print handoff increases administrative overhead and hassle the system can integrate a Two factor authentication system which requires users to provide a digital token (The generated Job ID) and student ID card to receive their printout

3.4 External Interfaces Requirements

This section describes how each interface should be designed and what functionalities should they fulfill

3.4.1 User Interface

The system shall provide a responsive, web-based interface designed with an approach to accommodate students submitting jobs via smartphones and laptops

- Intuitive Design: UI shall utilize a clean, minimal aesthetic to ensure that first-time users can navigate from file upload to payment in minimal clicks or interactions

- Modules: The interface must feature three distinct zones: a Unified Upload Section (supporting drag-and-drop), an Intelligent Instruction Input (dropdowns + NLP text field), and a Personal Job Tracking Dashboard

3.4.2 Hardware Interface

- Printer Connectivity: The system shall interface with the college's network-connected printers via standard IP-based printing protocols. It must be capable of transmitting processed print data over the LAN

3.4.3 Software Interface

- Cloud Integration: The system shall communicate with Cloud Storage APIs (primarily Google Drive) using OAuth 2.0 protocols to allow secure file access without storing user credentials
- Payment Integration: The system shall interface with the Razorpay API to manage the entire payment lifecycle
- Backend Communication: A dedicated bridge shall exist between the Node.js backend and the Python-based NLP module, ensuring that instruction strings are passed and parsed parameters are returned with minimal latency.

3.4.4 Communication Interface

- Secure Protocols: All data exchange between the client browser and the server shall be encrypted using HTTPS
- API Communication: The system shall utilize Secure API communication for all third-party interactions, employing API keys and webhooks to ensure that sensitive data, such as payment confirmations, cannot be intercepted

3.5 Non Functional Requirements

3.5.1 Performance

- Optimized Upload Speeds: The system shall utilize asynchronous processing to ensure that file uploads and metadata extraction do not block the user interface. Files under 10MB shall be processed within 5 seconds

- NLP latency: The Python-based NLP module shall parse and return structured parameters for natural language instructions within a maximum latency of 2 seconds
- Concurrent Queuing: The queue management system shall be capable of updating job statuses across all user dashboards in near real-time (under 1 second) to ensure transparency

3.5.2 Reliability

- Data Integrity: The system shall utilize database transactions to ensure that no print job is "lost" between payment and queuing. If a payment is verified, a corresponding job record must exist
- Recovery and fault tolerance: In the event of a server crash, the system shall be capable of restoring the print queue from the last known state in the MongoDB database upon restart. No paid job shall require a second payment due to system failure

3.5.3 Security

- Document Privacy: all uploaded files shall be encrypted. To ensure privacy, the source files shall be automatically deleted from temporary storage as soon as job is completed and at the server storage within 1 month of job completion.
- Transaction Encryption: All communication between the user, the server, and the Razorpay gateway shall be encrypted using SSL/TLS (HTTPS) protocols. No sensitive financial credentials shall be stored on college-side servers.
- User Authentication: Each Users login will be done using secure login methods to prevent unauthorized access to user print histories and balances.

3.5.4 Usability

- Intuitive Navigation: The user interface shall be designed such that a first-time user can submit a print job and complete payment in under 90 seconds without external guidance.
- Feedback Loops: The system shall provide immediate visual feedback for every user action, such as file upload progress bars, payment success pop-ups, and live "Position in Queue" updates.
- Accessibility: The web application shall be responsive, maintaining full functionality on both high-resolution desktop monitors and mobile device screens

3.5.5 Scalability

- **Concurrent User Support:** The MERN-stack architecture shall be configured to handle at least 100 concurrent user sessions during peak hours without a significant increase in response time.
- **Hardware Extensibility:** The backend shall be designed to be "printer-agnostic," allowing administrators to add new network-connected printers to the system via the dashboard without requiring changes to the core codebase.

4 Software Design Description

4.1 Introduction

The purpose of this Software Design Document (SDD) is to provide a detailed technical architecture for PrintEase, a web-based automated printing system designed to modernize college printing workflows. This document serves as the primary bridge between the requirements defined in the Software Requirements Specification Section (SRS) and the actual implementation of the system. It outlines the high-level system design, the decomposition of modules such as the Natural Language Processing (NLP) engine and the MERN-stack backend, and the data structures required to manage print jobs and secure financial transactions. By defining the software components, interfaces, and database schema, this SDD ensures that the developer team has a clear, actionable blueprint for building a scalable and reliable institutional printing solution.

4.1.1 Definitions, Acronyms, and Abbreviations

Term	Description
SDD	Software Design Document
SRS	Software Requirements Specification
NLP	Natural Language Processing
MERN	MongoDB, Express.js, React, Node.js
UI	User Interface
FIFO	First-In-First-Out
ID	Identification Document

DFD	Data Flow Diagram
ER	Entity Relationship

4.2 System Architecture

For PrintEase, we decided to use a function-oriented design approach. This choice is driven by the fact that the system's operations can be logically decomposed into distinct subsystems that interact through continuous data transfer and feedback loops. The architecture is partitioned into the following high-level subsystems:

- **User/Admin UI Subsystem:** This serves as the primary interface where students upload documents and provide instructions, and where administrators monitor the active print queue and manage hardware status.
- **Backend Logic Subsystem:** This acts as the central processing hub, responsible for executing cost calculation algorithms, maintaining the FIFO (First-In-First-Out) queue, making calls to the Razorpay payment gateway, and performing all necessary database queries.
- **Database & Storage Subsystem:** This subsystem provides persistent storage for user profiles and transaction logs while also managing the temporary, encrypted storage of uploaded files until the print job is successfully completed.

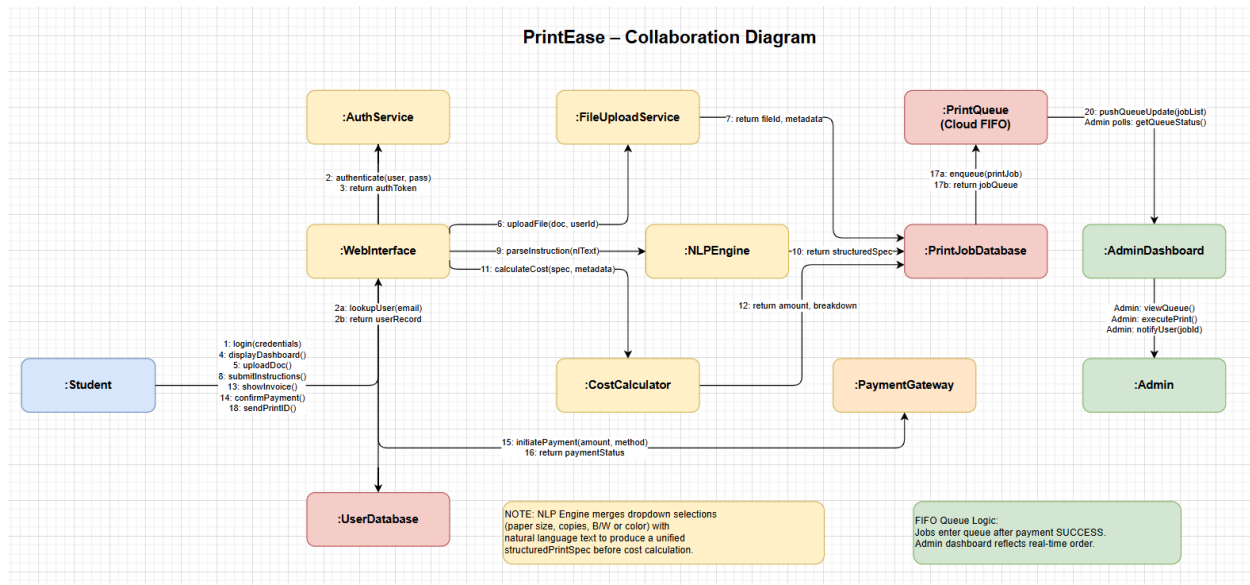


Figure 1: Collaboration Diagram

The collaboration diagram depicts the interaction between different system components in terms of message flow. It focuses on how objects such as the user, authentication service, file upload service, cost calculator, and payment system communicate with each other. Each interaction is numbered to indicate the sequence of operations. It complements the sequence diagram by providing a more network-oriented view of system interactions.

4.2.1 Decomposition Description

To Explain these subsystems in further detail we used Data Flow Diagrams (DFD)

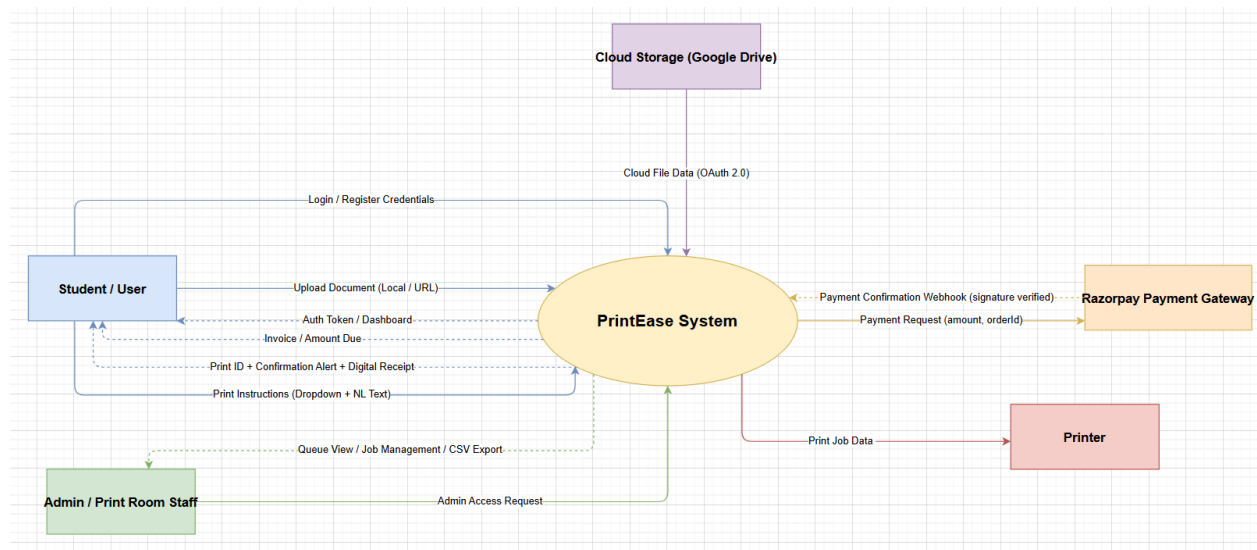


Figure 2: Level 0 Data Flow Diagram

The Level 0 Data Flow Diagram represents the entire PrintEase system as a single high-level process interacting with external entities.

The primary external entities include the Student/User, Admin/Print Room Staff, Razorpay Payment Gateway, Cloud Storage (Google Drive), and the Printer. The system receives inputs such as login credentials, document uploads, and print instructions from the user. It processes these inputs and returns outputs such as authentication responses, invoice details, print confirmations, and notifications.

Additionally, the system interacts with external services such as the payment gateway for transaction processing and cloud storage for document retrieval. The admin interacts with the system for queue monitoring and print execution. This diagram establishes the system boundary and clearly defines all external interactions.

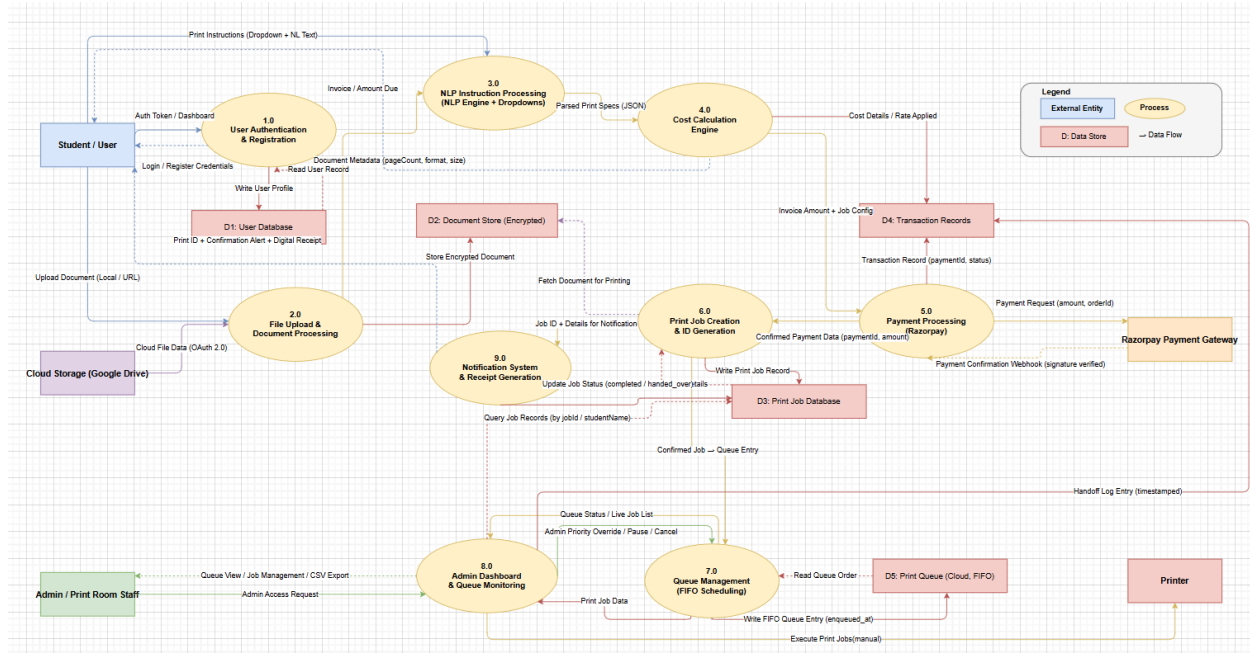


Figure 3: Level 1 Data Flow Diagram

The Level 1 Data Flow Diagram expands the PrintEase system into its major functional processes and illustrates how data flows between them. The system is decomposed into nine core processes: authentication, file upload, NLP instruction processing, cost calculation, payment processing, print job creation, queue management, admin dashboard, and notification system.

Users interact with the system by uploading documents and specifying print instructions. These inputs are processed through NLP modules to extract structured print parameters, which are then used by the cost calculation engine to generate an invoice. Once payment is successfully completed via Razorpay, a print job is created and stored in the database.

The system maintains multiple data stores such as the user database, document storage, print job database, transaction records, and print queue. The queue management system ensures FIFO scheduling of print jobs. The admin dashboard provides visibility and control over the queue, enabling manual intervention if and when required.

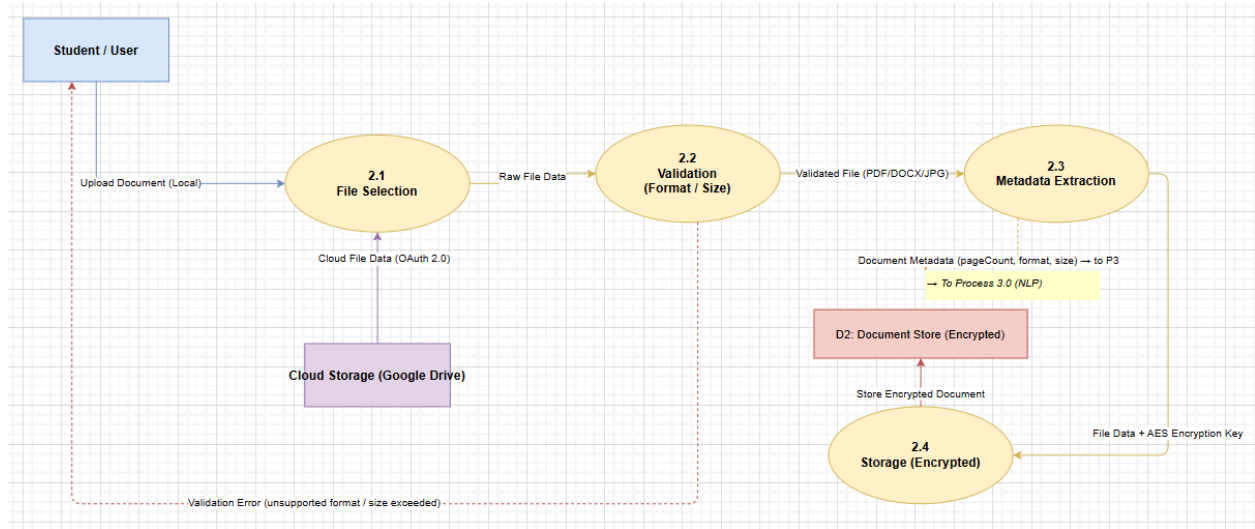


Figure 4.1: Level 2 Data Flow Diagram-File Upload and Document Processing

This Level 2 DFD decomposes the File Upload and Document Processing module into detailed sub-processes. The process begins with file selection, where the user uploads a document either locally or from cloud storage. The uploaded file is then validated for format and size constraints to ensure compatibility.

Following validation, metadata such as page count, file format, and size is extracted. This metadata is crucial for subsequent processes such as NLP instruction processing and cost calculation. The document is then securely stored in the encrypted document store.

This module ensures data integrity, security, and proper preprocessing of documents before they are passed to downstream components.

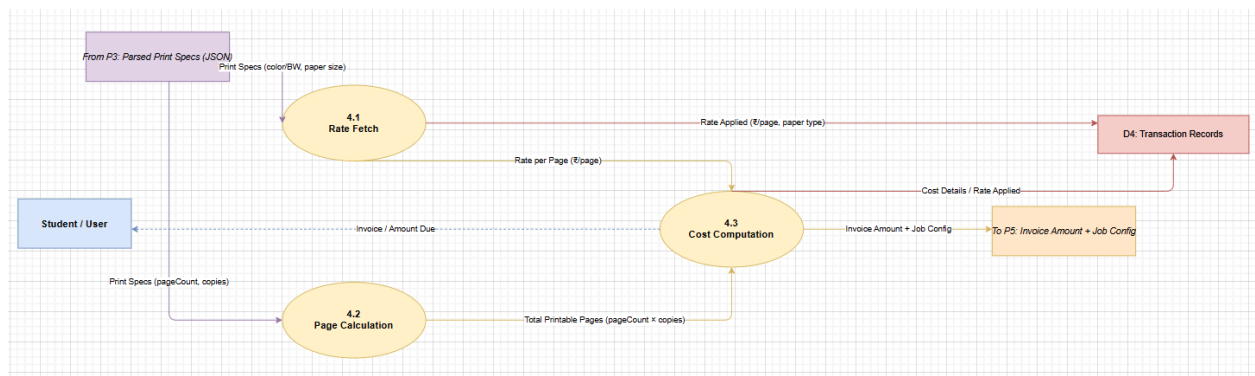


Figure 4.2: Level 2 Data Flow Diagram-Cost Calculation Engine

The Cost Calculation module is responsible for computing the total printing cost based on user-defined parameters and document characteristics. It consists of three main sub-processes: rate fetching, page calculation, and cost computation.

The system first retrieves applicable pricing rates based on parameters such as color mode, paper size, and printing type. It then calculates the number of pages and applies relevant multipliers. Finally, the total cost is computed and formatted into an invoice.

The computed cost is stored in transaction records and forwarded to the payment processing module. This ensures accurate billing and transparency for the user.

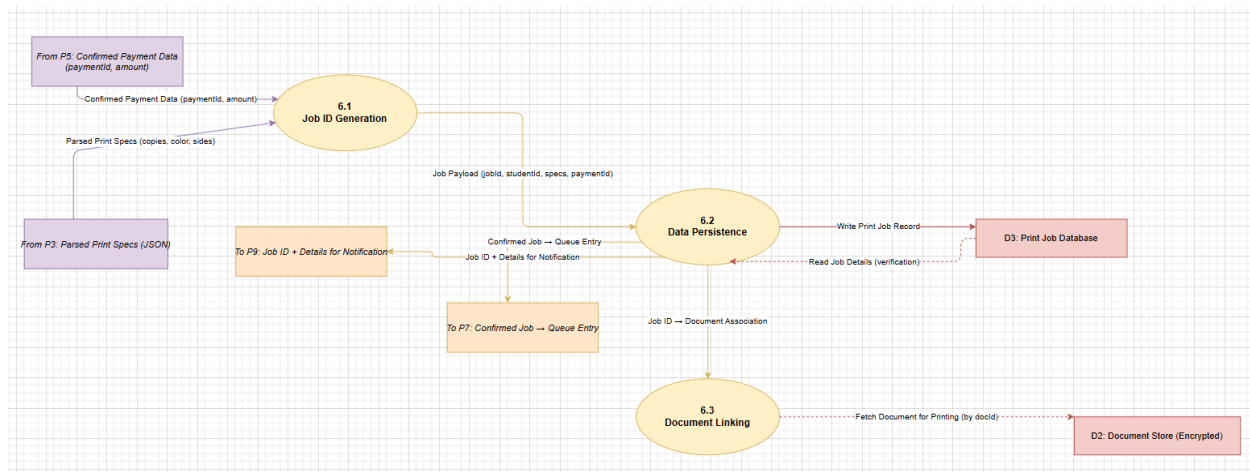


Figure 4.3: Level 2 Data Flow Diagram-Print Job Generation

This diagram details the Print Job Creation and ID Generation process, which is triggered after successful payment confirmation. The system generates a unique job ID to identify each print request.

The process integrates data from multiple sources, including parsed print specifications and payment confirmation details. These are consolidated and stored in the print job database. The system also retrieves the corresponding document from the document store for printing.

Once the job is created, it is forwarded to the queue management system for scheduling and to the notification system for informing the user. This module acts as a bridge between payment processing and actual print execution.

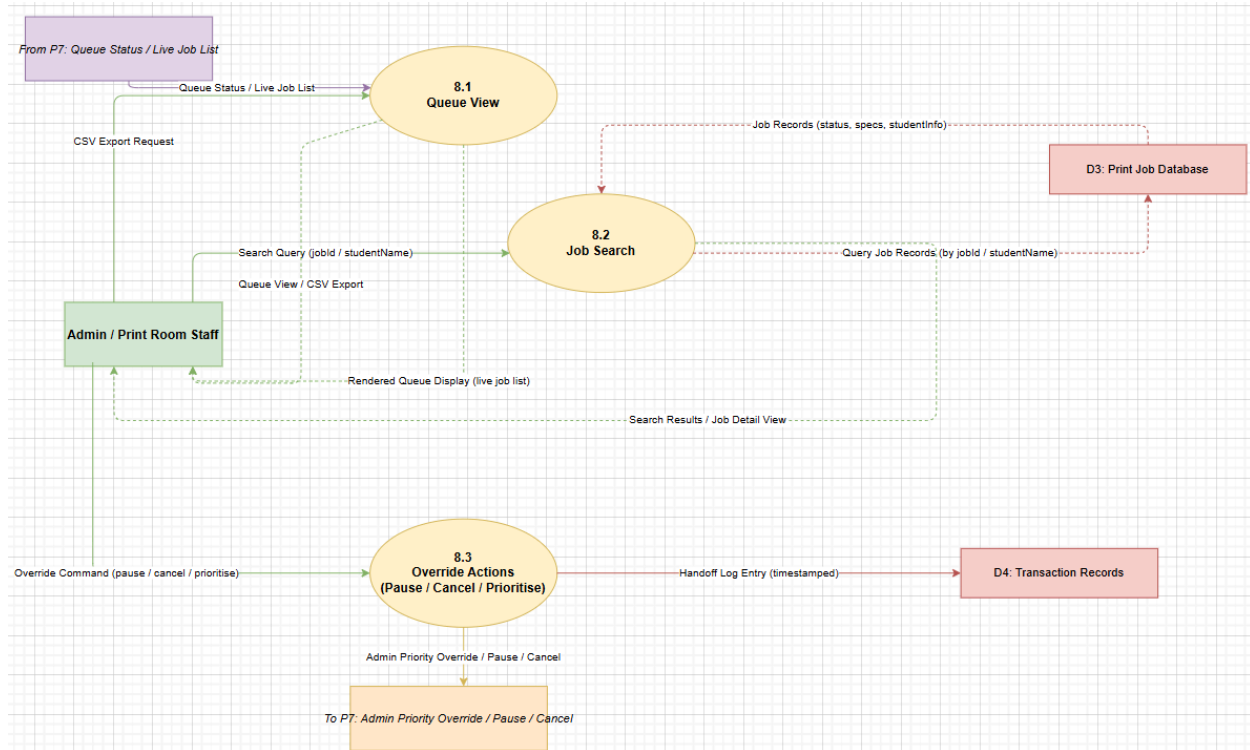


Figure 4.4 Level 2 Data Flow Diagram- Admin Dashboard and Queue Monitoring

The Admin Dashboard module provides an interface for print room staff to monitor and control the printing workflow. It is decomposed into three sub-processes: queue view, job search, and override actions.

The queue view allows admins to monitor real-time job status and ordering. The job search functionality enables the retrieval of specific jobs based on identifiers such as job ID or user details. The override actions module allows admins to pause, cancel, or reprioritize jobs in the queue.

This module interacts with the print job database and transaction records for data retrieval and logging. It also communicates with the queue management system to apply changes. The admin ultimately executes the printing process manually, ensuring operational control and flexibility.

4.2.2 Design Rationale

Building the PrintEase system with the MERN stack provides a natural technical boundary between each subsystem, making the functional decomposition clear based on the specific technologies used.

- Frontend: By using React, the User/Admin UI is physically and logically separated from the server logic. It is used specifically to handle the "Client-Side" responsibilities, such as

managing the document upload interface and providing real-time dashboard updates without reloading the page.

- **Backend:** The Backend Logic Subsystem is defined by the Node.js runtime and Express framework. This tech is used because its asynchronous nature is ideal for handling the high-volume functional calls required for cost calculation, FIFO queue management, and interacting with the Razorpay API.
- **Database:** The Database Subsystem is built using MongoDB, which serves as the persistent data layer. It is used because its document-oriented structure easily handles the varied metadata associated with print jobs, user profiles, and transaction logs described in the system requirements

4.3 Data Description

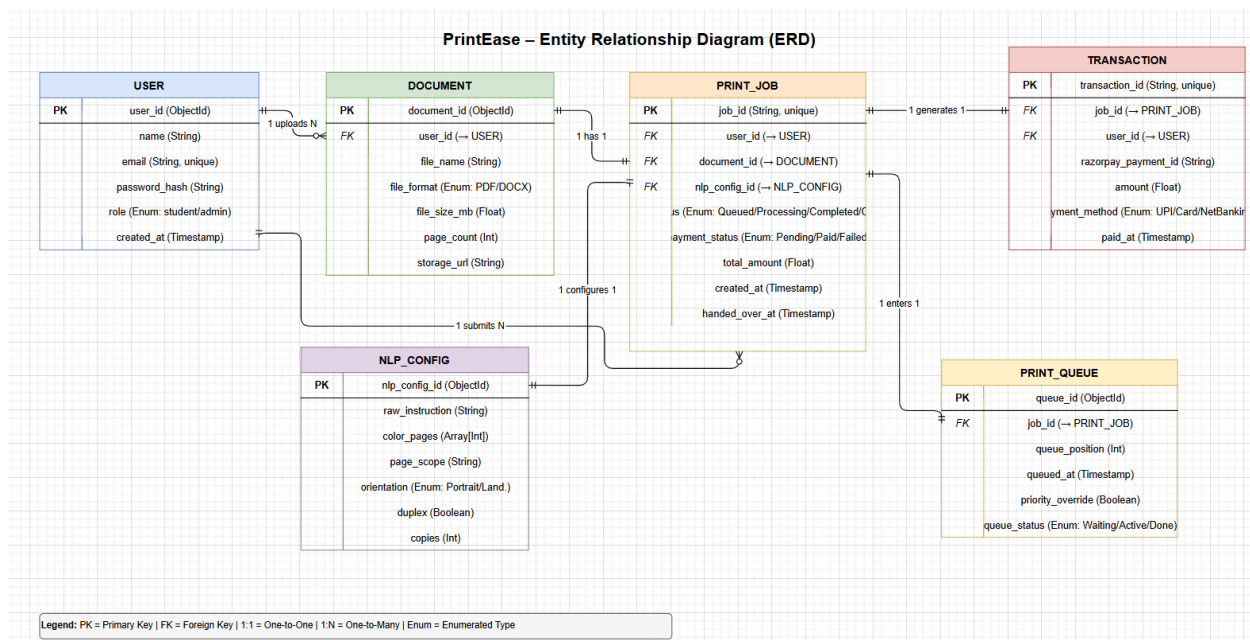


Figure 5: Entity Relationship Diagram

The Entity Relationship Diagram represents the database structure of the PrintEase system. It defines key entities such as User, Document, Print Job, and Transaction, along with their attributes and relationships.

Each entity includes primary keys and relevant attributes. Relationships between entities are established using foreign keys, such as linking documents and print jobs to users. The design ensures data normalization and avoids redundancy.

4.4 Component Interaction

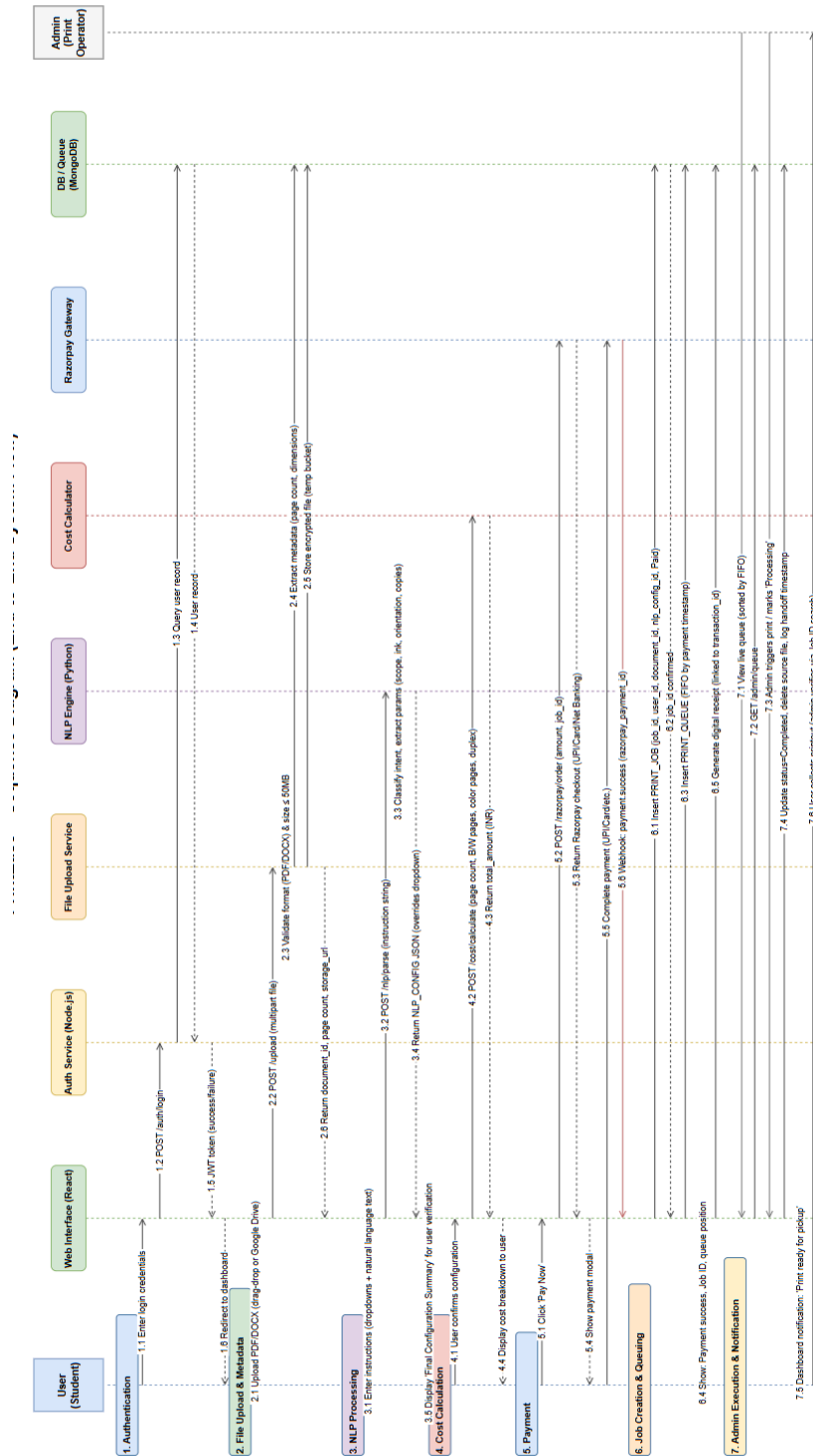


Figure 6: Sequence diagram

The sequence diagram illustrates the chronological interaction between system components during the execution of a print request. It begins with user authentication and proceeds through file upload, NLP processing, cost calculation, and payment processing.

Once the payment is confirmed, the system creates a print job and inserts it into the queue. The admin monitors the queue and executes the printing process. Throughout this flow, different services such as the web interface, backend services, NLP engine, and payment gateway interact in a coordinated manner.

4.5 User Interface

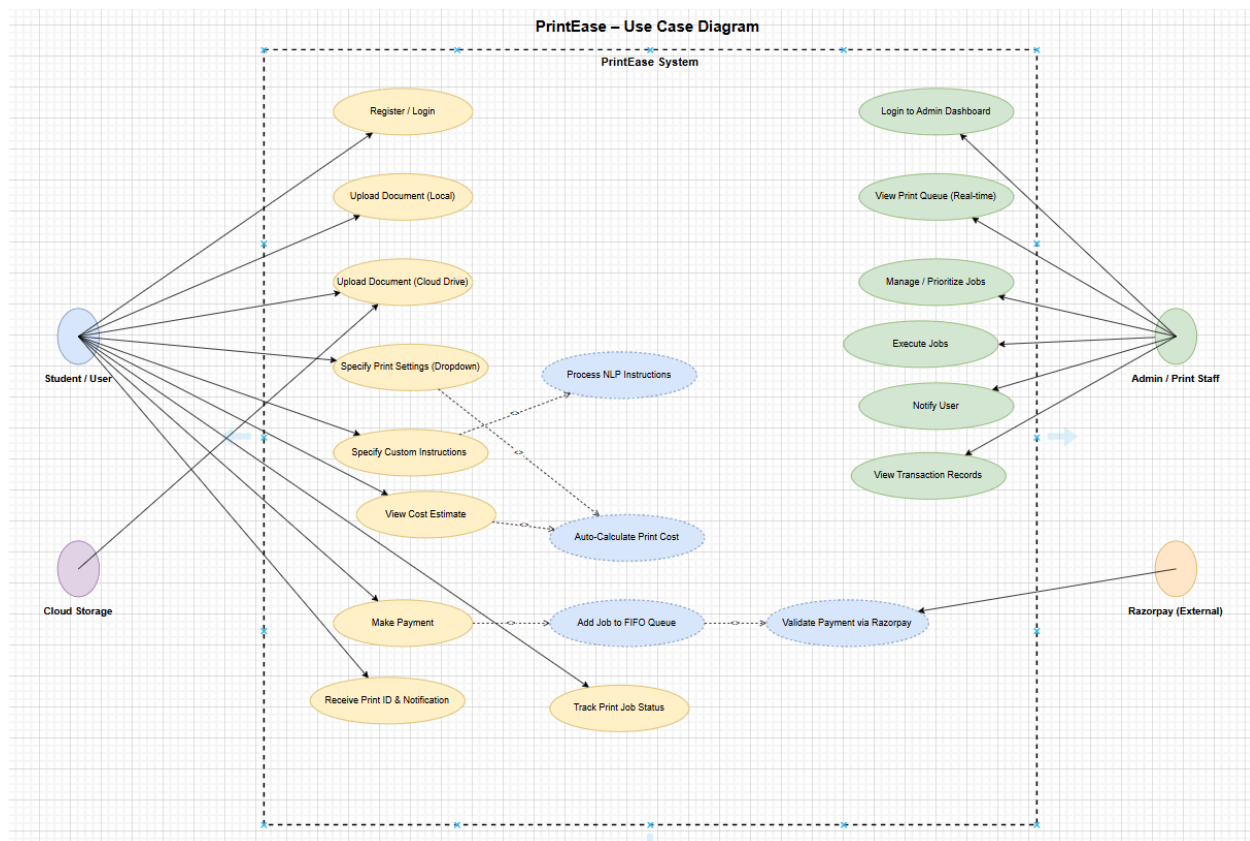


Figure 7: Use case diagram

The Use Case Diagram illustrates the functional interactions between users and the system. It identifies key actors such as the Student/User, Admin, Payment Gateway, and Cloud Storage.

The diagram includes use cases such as registration, login, document upload, specifying print settings, making payments, and managing print jobs. Admin-specific use cases include queue monitoring and job management.

4.6 Project Schedule

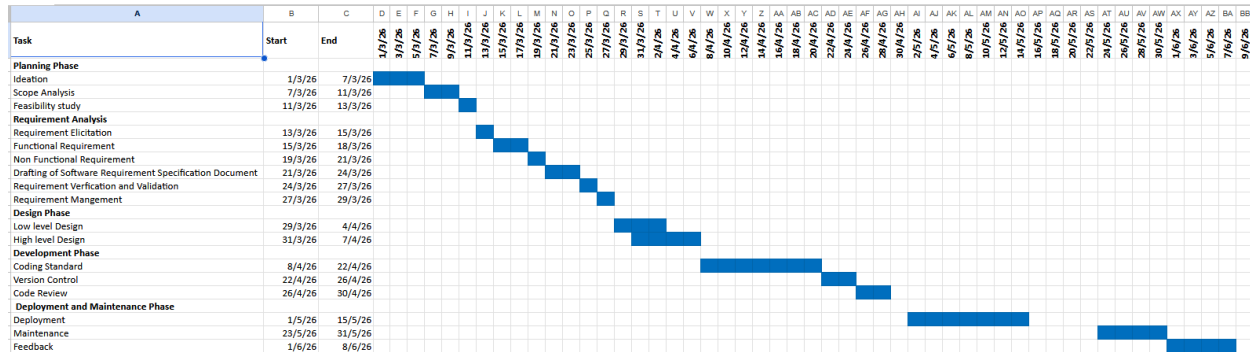


Figure 8: Gantt Chart

The Gantt chart above shows the project schedule we aimed to follow

5 Implementation

The implementation phase of the PrintEase project focuses on converting the system design and requirements into a fully functional web-based printing management application. The project was developed using a modern full-stack web development approach where the frontend handles user interaction and the backend manages authentication, file handling, database operations, and payment processing.

The implementation was carried out in a modular and scalable manner so that each component of the application can be maintained and upgraded independently. The system provides students and users with a convenient platform to upload documents, customize print options, place printing requests, and securely complete payments online.

5.1 Tech stack

Technology	Purpose
React.js	Frontend user interface development
Vite	Fast frontend build tool and development server
Tailwind CSS	Responsive and modern ui styling
Node.js	Backend runtime environment
Express.js	REST API development

MongoDB	Database for storing users and print requests
Mongoose	MongoDB Object Modelling
JWT	User Authentication and Authorization
Multer	File upload Handling
Razorpay	Online payment integration
Axios	API communication
React Router DOM	Client-side routing
Zod	Input validation
bcryptjs	Password hashing and security

5.1.1 Frontend Technologies

The frontend of PrintEase is developed using React.js with Vite. React enables component-based development, making the user interface modular and reusable. Vite improves the development experience by providing faster startup and optimized build performance.

Tailwind CSS is used for designing the interface. It helps create a responsive and clean user interface with minimal custom CSS. React Router DOM is used for page navigation without reloading the application.

5.1.2 Backend Technologies

The backend is developed using Node.js and Express.js. Express is used to create RESTful APIs that handle user authentication, file uploads, order management, and payment verification.

MongoDB is used as the database because of its flexibility and document-oriented structure. Mongoose simplifies database operations and schema management.

5.1.3 Security and Authentication

User passwords are encrypted using bcryptjs before storing them in the database. JWT authentication is implemented to maintain security and reliability.

5.1.4 Payment Integration

The application integrates Razorpay SDK for secure online transactions. Users can pay for print orders digitally before confirming the request.

5.2 Key Algorithms/ Logic

5.2.1 Authentication Logic

Authentication is implemented using JWT-based authentication

Core Logic of the Algo :

1. The user enters an email and password.
2. Passwords are encrypted using bcryptjs.
3. User credentials are validated.
4. JWT token is generated.
5. Protected routes are accessed only with valid tokens.

5.2.2 File Upload Logic

The application supports uploading PDF documents for printing.

Core Logic of the Algo :

1. The user selects a PDF file.
2. Multer middleware handles file uploads.
3. File type validation is performed.
4. File is temporarily stored and processed.
5. Metadata is saved in MongoDB.

5.2.3 Print Cost Calculation Logic

The application automatically calculates the printing cost based on selected print options.

1. Calculate B/W Cost: it takes the number of B/W pages and multiplies them by 2
2. Calculate Colored Cost: It takes the number of colored pages and multiplies them by 6.
3. Sum the Totals: It adds the results of the black-and-white cost and the colored cost together.
4. Return the Result: The final combined total is sent back (returned) as the output of the function.

Assuming that the cost if black and white print is Rs. 2 and for colored it is Rs. 6.

5.2.4 Payment Processing Logic

The payment system ensures secure digital transactions.

1. User clicks on proceed to payment.
2. The payment gateway window opens.
3. The user completes the transaction.
4. Payment status is verified.
5. Request is confirmed only after successful payment.

5.2.5 Request Management Logic

The admin or manager can manage print requests from the dashboard

5.3 Database implementation



Figure 9: Tables in Database

```

_id: ObjectId('69de8d2228e56f3e8b29de3f')
userId: ObjectId('69de8cc728e56f3e8b29de2f')
items: Array (3)
current: Array (3) {}
totalAmount: 18
status: "completed"
paymentStatus: "paid"
paymentProvider: "razorpay"
paymentRef: "pay_SdTkWcBT2IGMGK"
paymentDetails: Object
notifications: Array (1)
  0: Object
    createdAt: 2026-04-14T18:53:22.736+00:00
    updatedAt: 2026-04-14T18:56:48.765+00:00
    __v: 1
    paidAt: 2026-04-14T18:54:32.986+00:00

_id: ObjectId('69de8d7128e56f3e8b29de47')
userId: ObjectId('69de8cc728e56f3e8b29de2f')
items: Array (3)
currency: "INR"
totalAmount: 18
status: "pending"
paymentStatus: "pending"
paymentProvider: "none"
paymentRef: ""
paymentDetails: Object
notifications: Array (empty)
  createdAt: 2026-04-14T18:54:41.554+00:00
  updatedAt: 2026-04-14T18:54:41.554+00:00
  __v: 0
  
```

Figure 10: Order Sample as received by the admin.

```

_id: ObjectId('69de8ce428e56f3e8b29de34')
ownerUserId : ObjectId('69de8cc728e56f3e8b29de2f')
originalName : "Screenshot 2026-04-10 at 12.34.02â€ˆAM.png"
mimeType : "image/png"
sizeBytes : 0
storage : "disk"
diskPath : ""
pageCount : 1
createdAt : 2026-04-14T18:52:20.188+00:00
updatedAt : 2026-05-06T13:05:49.973+00:00
__v : 0
deletedAt : 2026-05-06T13:05:49.967+00:00

```

Figure 11: Stored files in db

```

{
  "_id": {"_id": "69de8ce428e56f3e8b29de34"},
  "name": "Harshit",
  "email": "241210047@nitdelhi.ac.in",
  "phone": "1234567890",
  "rollNo": "241210047",
  "passwordHash": "$2a$10$RxyoCCMe4up4wB.rmHobBe0ezI5xc50vyXJ71nvMD4d6/UyWccF1q",
  "role": "admin",
  "createdAt": {"_id": "69de8ce428e56f3e8b29de34"},
  "updatedAt": {"_id": "69de8cc728e56f3e8b29de2f"},
  "__v": 0
}

```

Figure 12 :Users in db example

5.4 Product Screenshots

5.4.1 Login Page

The Login Page is the entry point of the PrintEase application that allows registered users to securely access the system using their email and password credentials. The page is designed with a simple and user-friendly interface to ensure smooth authentication for users.

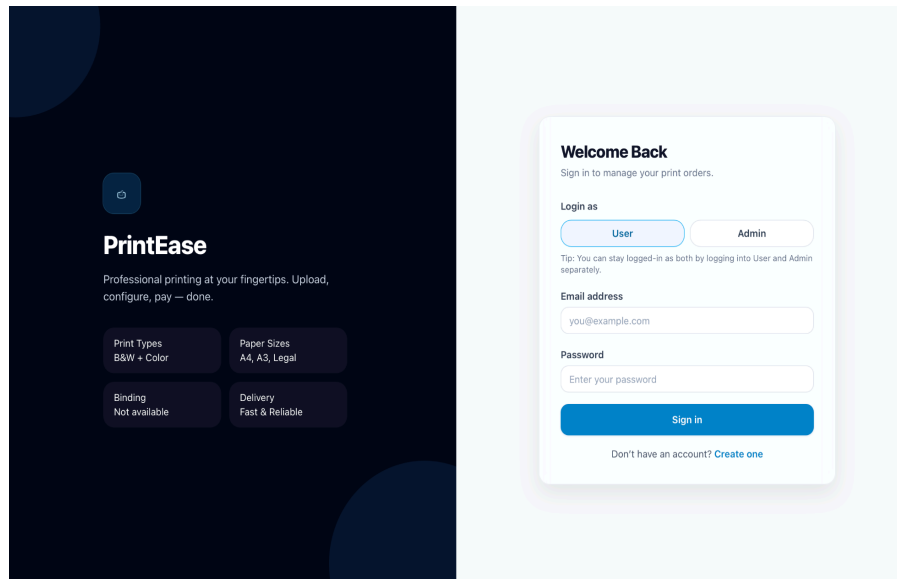


Figure 13 : Login Page

5.4.2 Home Dashboard

The Home Dashboard is the main interface of the PrintEase application where users can access all primary printing functionalities. After successful login, users are redirected to this dashboard, which provides an organized and interactive environment for managing print requests.

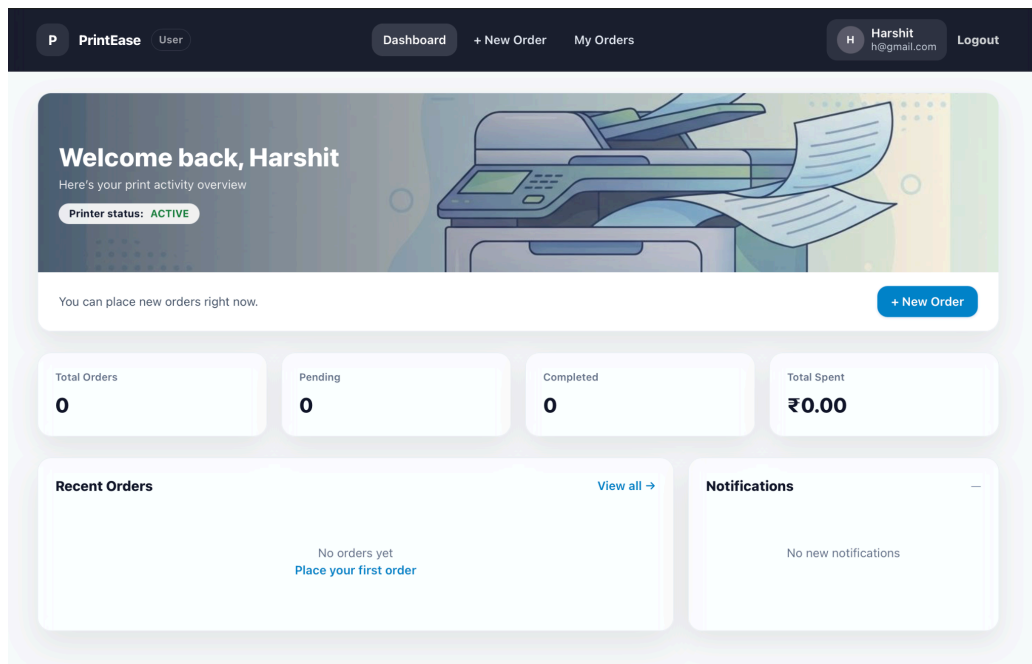


Figure 14: User Dashboard

5.4.3 PDF Upload Section

The PDF Upload Section allows users to upload documents securely for printing. This module is one of the core functionalities of the PrintEase application, enabling users to select PDF files directly from their device and submit them for print processing.

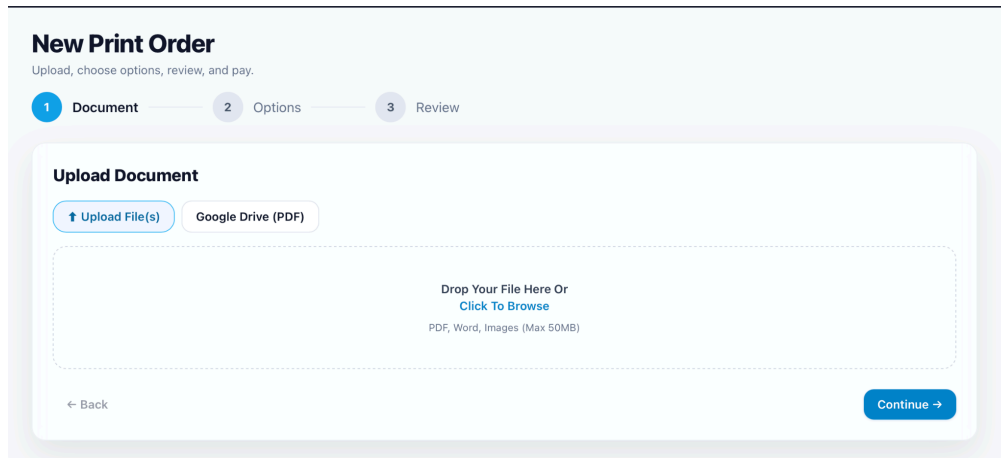


Figure 15: Upload Docs

5.4.4 Print Details Page

After selecting print preferences and calculating the total cost, users are redirected to the payment page to complete the transaction.

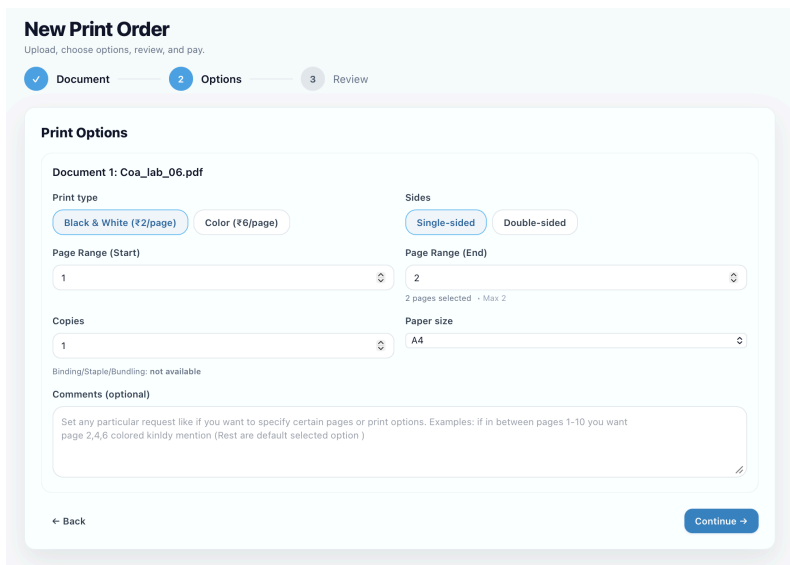
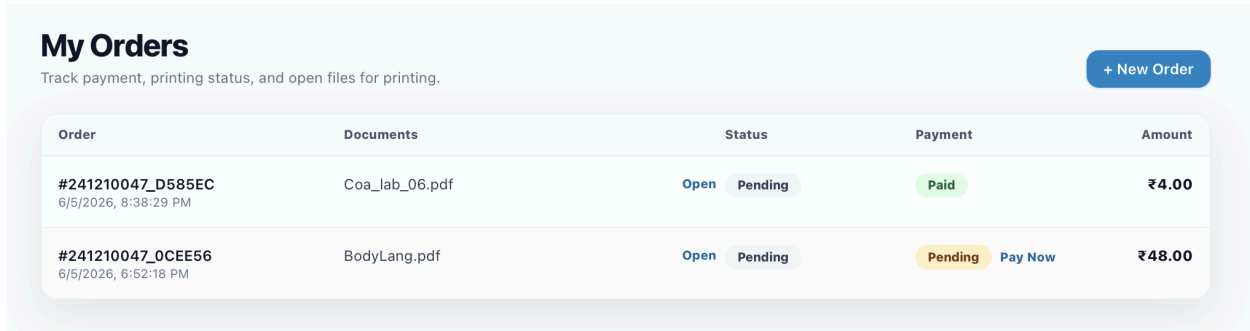


Figure 16: Print details

5.4.5 My Orders Page

The My Orders Page allows users to view and track all their submitted print requests in one place. This section helps users monitor the status of their orders, payment details, and printing progress in real time. Each order entry displays important information such as uploaded document details, print type, payment status, request status, and timestamps. Users can easily identify whether a request is pending, in progress, or completed.



My Orders
Track payment, printing status, and open files for printing. [+ New Order](#)

Order	Documents	Status	Payment	Amount
#241210047_D585EC 6/5/2026, 8:38:29 PM	Coa_lab_06.pdf	Open Pending	Paid	₹4.00
#241210047_OCEE56 6/5/2026, 6:52:18 PM	BodyLang.pdf	Open Pending	Pending Pay Now	₹48.00

Figure 17: My orders

5.4.6 Admin Dashboard

The Admin Dashboard is a dedicated management interface designed for administrators to monitor and control all printing activities within the PrintEase system. This dashboard provides complete visibility of user orders, payment records, printer status, and document processing workflows.

The system includes revenue management features that help administrators track daily earnings, verify payments, and generate payment records. Search and filtering functionalities are also implemented to quickly locate specific orders or users.

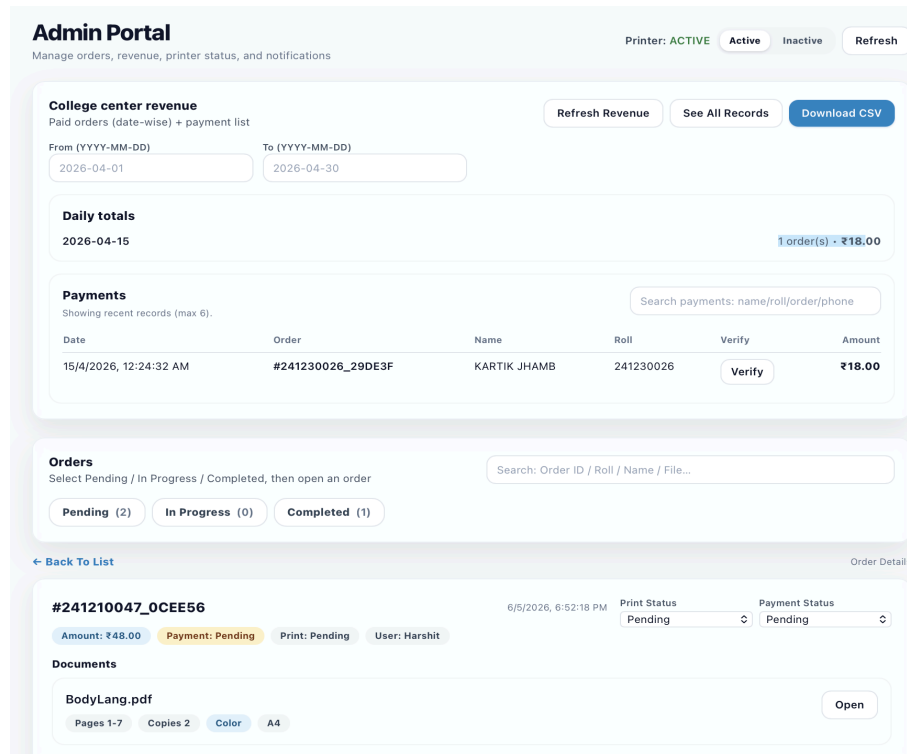


Figure 18: Admin Dashboard

5.4.7 Order Summary Screen for Admin

The Order Summary Screen for Admin provides a detailed overview of an individual print request submitted by a user. This screen helps administrators monitor, verify, and manage the complete lifecycle of a printing order from submission to completion.

The screen displays important order information such as order ID, payment amount, payment status, print status, user details, uploaded document information, page range, copy count, print type, and paper size. Administrators can easily review the uploaded document before processing the print request.

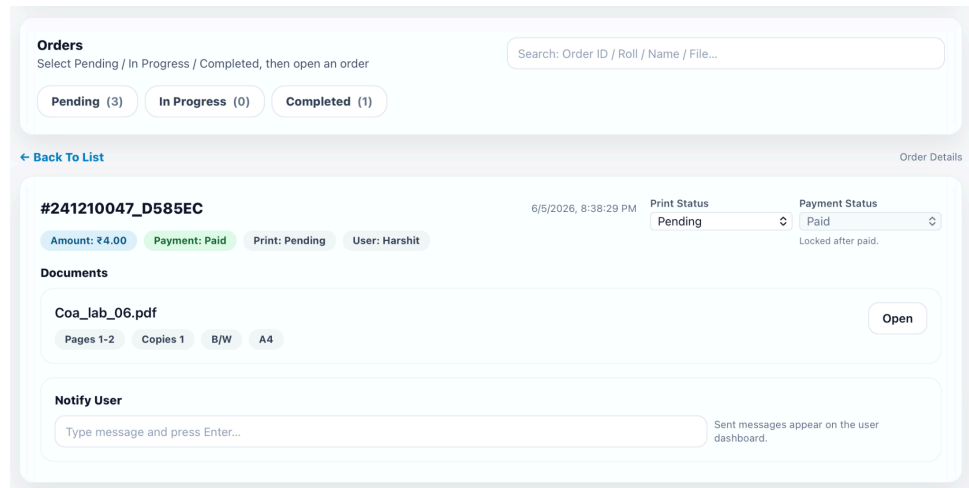


Figure 19: Order Summary

5.4.8 Payment Portal

The Payment Page is responsible for handling secure online transactions for print requests. Here users can pick how they would like to pay via different payment methods like Net banking, UPI, cards and wallets.

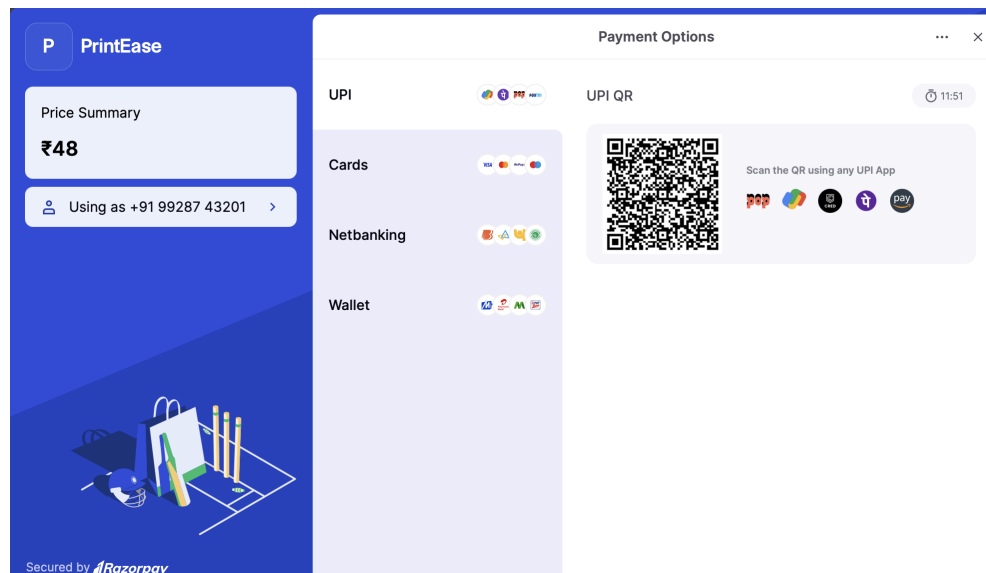


Figure 20: Payment Portal

6. Testing

6.1 Testing Strategies

- Unit testing: isolates each functional module, like the cost calculation engine, to verify its logic independently. This ensures core algorithms, such as the NLP parameter override, produce correct outputs for every input before integration.
- Integration testing: evaluates data exchange between subsystems, such as the Node.js backend and Razorpay API. It confirms that "handshakes" are successful, like verifying payment webhooks accurately update the MongoDB database.
- System testing provides an end-to-end evaluation of the complete application against Test cases defined below
- Performance Testing: Alongside System testing performance testing such as performance, usability and security were conducted

All of the testing strategies were done manually

6.2 Testing Cases

Based on the Functional requirements(FR), Non-functional requirements(NR), and External Interface requirements (ER)described above, the following are tests that will be used to identify if defined requirements are being fulfilled with the designed system.

TC #	Acceptance Criteria (Test Case)	Requirements Addressed
TC 1	A user can successfully upload a 5MB PDF, enter NLP instructions, and complete a payment within 90 seconds.	FR: 3.1, 3.3, 3.4 IR: 4.1, 4.3 NR: 5.4
TC 2	The system rejects a 60MB file or an unsupported format (e.g., .PNG) with a clear error message and prevents upload.	FR: 3.1 NR: 5.1

TC 3	The system correctly extracts specific parameters (quantity, scope, ink used) from a text string with 90%+ accuracy.	FR: 3.3 IR: 4.3
TC 4	If a user selects "B/W" in a dropdown but types "Print in color" in the text box, the final cost calculation reflects color pricing.	FR: 3.3, 3.4
TC 5	A job is only visible in the "Active Queue" after a successful Razorpay webhook is received; failed payments do not enter the queue.	FR: 3.4, 3.6 NR: 5.2, 5.3
TC 6	The user dashboard and admin dashboard reflect the same status (e.g., "Processing") within 1 second of a status change.	FR: 3.2, 3.5, 3.7 NR: 5.1
TC 7	The admin can successfully search for a Job ID and mark it "Handed Over," which immediately triggers the deletion of the source file.	FR: 3.5, 3.7 NR: 5.3
TC 8	After a simulated server crash and restart, all "Paid" jobs are successfully reloaded into the queue in their original FIFO order.	FR: 3.5, 3.6 NR: 5.2
TC 9	The interface remains fully functional and visually consistent across both mobile and laptop operating Systems	IR: 4.1 NR: 5.4
TC 10	The system successfully processes 100 simultaneous "Transaction History" requests without exceeding a 3-second response time.	FR: 3.2 NR: 5.5
TC 11	A user can successfully import a file from Google Drive via the UI without being asked for manual credentials twice.	FR: 3.1 IR: 4.3

TC 12	Admin Override Control: An admin can manually move a "Pending" job to the top of the queue or adjust settings successfully via the dashboard.	FR: 3.6, 3.7
-------	---	--------------

7. Conclusion And Future Scope

7.1 Conclusion

Through this project, our team successfully translated theoretical software engineering principles into a functional, real-world application, gaining invaluable insight into the complexities of the development lifecycle. The systematic application of diagrams enabled critical thinking that allowed us to identify logical bottlenecks early and streamline our testing and debugging processes.

While we achieved a comprehensive digital workflow, the final physical integration with college hardware remains a challenge due to the technical difficulties. Consequently, direct hardware execution has been categorized under future scope to allow for more dedicated research. Despite this, the current iteration of PrintEase achieves its primary goal: it replaces a fragmented, manual system with a unified MERN-stack platform that significantly reduces administrative overhead and provides students with a seamless, remote-access interface.

7.2 Future Enhancements

- **Mobile application support**
A dedicated Android and iOS application for PrintEase would significantly improve accessibility for students who primarily use smartphones. The app would have push notifications to alert users in real time when their print job is ready, removing the need to actively monitor the dashboard. Native mobile features such as camera-based document scanning will allow students to photograph handwritten notes and submit them as print jobs directly.
- **AI-based print optimization**
An AI-driven optimization layer could analyze submitted documents and automatically suggest cost-saving configurations, such as recommending duplex printing or grayscale for drafts. It could also detect redundant or duplicate print submissions from the same user, preventing unnecessary wastage of paper and ink. This enhancement would make PrintEase not only smarter but also contribute meaningfully to the college's sustainability goals.

- Institutional Analytics and Reporting Dashboard

The comprehensive analytics module would give college administration deeper insights into printing trends, resource consumption, and revenue patterns over time.

Automatically generated weekly and monthly reports in PDF or CSV format could track metrics such as peak usage hours, most common print configurations, and per-department spending. These insights could help management make data-driven decisions about printer procurement, pricing adjustments, and operational staffing. Integration with the college's existing ERP or finance systems could further streamline budget reconciliation and auditing processes.

- Loyalty and Prepaid Wallet System

A built-in digital wallet would allow students to preload a balance into their PrintEase account, reducing a fresh payment for every single print job. Faculty members could receive institutionally funded printing credits directly into their wallets, eliminating the need for reimbursement workflows. This system would also reduce dependency on third-party payment gateway availability for every individual transaction, improving overall system reliability.

8. References and Bibliography

- MongoDB, Inc. MongoDB Documentation. MongoDB, 2026, www.mongodb.com/docs/.
- Node.js Foundation. Node.js v20.x Documentation. Node.js, 2026, nodejs.org/api/.
- React Open Source Team. React Documentation: Quick Start. Meta Platforms, 2026, react.dev/learn.
- Razorpay Software Private Ltd. Razorpay Standard Integration Guide. Razorpay, 2026, razorpay.com/docs/payments/payment-gateway/.